```
print("Just right")
```

14.1 Other modules

Sometimes you want to use a python module that does not come with the Python installation. You can also import those, but you have to have them installed on your computer.

14.1.1 Creating your own module

When python reads the import command, it first checks files in your directory, then site-packages or pre installed modules. To make your own module, just create a .py file in the current directory and use the command:

```
import module
```

This will try to import the file module.py from your current directory and if not found, from site-packages and prepackaged modules. Changing module to the name of the .py file you created will import that file.

However, when it imports the module, it will basically start the file as a program, so any code on there will be run. You want to group all code into functions.

The `__name__` == `__main__` trick

In python, the variable `__name__` will give you the current name of the program. If a module you import prints the `__name__` variable, then it will print the name of the module. If the current file prints the `__name__` variable, it will print `__main__`, to show it is the main program.

If an if statement checks the name variable and runs code if the program is main, it can bypass the unintentional run problem created when a module is imported. Say for example you have a file, which runs some code. It also has a function you want to use in another program. However, you only want the function, not to run the code. By setting up the code below, it will only run the code if it is the file that was clicked on or started, not if it was imported.

```
if __name__ == '__main__':
    pass
```

In this instance, if the file is run but not imported, it will run the pass command. You can replace the pass command with the code you want to be run when not imported. Just remember to indent the code.

14.1.2 The pip module

The pip module is a module that comes with the python installation and acts as a module downloader/manager. You can download other modules from the internet with pip.

The pip module is not used in the python interpreter, but is run through the command line. To use it, open up your command line interpreter (for Windows it is Command Prompt, for Mac/Linux it is Terminal) and type in the following code:

```
py3 -m pip install module
```

or the alternate code

```
pip install module
```

This will try to download and install module from the user-submitted python modules database. Module can be changed to the name of the module.

15 More on Lists

We have already seen lists and how they can be used. Now that you have some more background I will go into more detail about lists. First we will look at more ways to get at the elements in a list and then we will talk about copying them.

Here are some examples of using indexing to access a single element of a list:

```
>>> some_numbers = ['zero', 'one', 'two', 'three', 'four', 'five']
»> some_numbers[0]
'zero'
»> some_numbers[4]
'four'
»> some_numbers[5]
'five'
```

All those examples should look familiar to you. If you want the first item in the list just look at index 0. The second item is index 1 and so on through the list. However what if you want the last item in the list? One way could be to use the `len()` function like `some_numbers[len(some_numbers) - 1]`. This way works since the `len()` function always returns the last index plus one. The second from the last would then be `some_numbers[len(some_numbers) - 2]`. There is an easier way to do this. In Python the last item is always index -1. The second to the last is index -2 and so on. Here are some more examples:

```
>>> some_numbers[len(some_numbers) - 1]
'five'
»> some_numbers[len(some_numbers) - 2]
'four'
»> some_numbers[-1]
'five'
»> some_numbers[-2]
'four'
»> some_numbers[-6]
'zero'
```

Thus any item in the list can be indexed in two ways: from the front and from the back.

Another useful way to get into parts of lists is using slicing. Here is another example to give you an idea what they can be used for:

```
>>> things = [0, 'Fred', 2, 'S.P.A.M.', 'Stocking', 42, "Jack", "Jill"]
»> things[0]
0
»> things[7]
'Jill'
»> things[0:8]
```

```
[0, 'Fred', 2, 'S.P.A.M.', 'Stocking', 42, 'Jack', 'Jill']
»> things[2:4]
[2, 'S.P.A.M.']
»> things[4:7]
['Stocking', 42, 'Jack']
»> things[1:5]
['Fred', 2, 'S.P.A.M.', 'Stocking']
```

Slicing is used to return part of a list. The slicing operator is in the form `things[first_index:last_index]`. Slicing cuts the list before the `first_index` and before the `last_index` and returns the parts in between. You can use both types of indexing:

```
>>> things[-4:-2]
['Stocking', 42]
»> things[-4]
'Stocking'
»> things[-4:6]
['Stocking', 42]
```

Another trick with slicing is the unspecified index. If the first index is not specified the beginning of the list is assumed. If the last index is not specified the whole rest of the list is assumed. Here are some examples:

```
>>> things[:2]
[0, 'Fred']
»> things[-2:]
['Jack', 'Jill']
»> things[:3]
[0, 'Fred', 2]
»> things[:-5]
[0, 'Fred', 2]
```

Here is a (HTML inspired) program example (copy and paste in the poem definition if you want):

```
poem = ["<B>", "Jack", "and", "Jill", "</B>", "went", "up", "the",
        "hill", "to", "<B>", "fetch", "a", "pail", "of", "</B>",
        "water.", "Jack", "fell", "<B>", "down", "and", "broke",
        "</B>", "his", "crown", "and", "<B>", "Jill", "came",
        "</B>", "tumbling", "after"]

def get_bold(text):
    true = 1
    false = 0
    ## is_bold tells whether or not we are currently looking at
    ## a bold section of text.
    is_bold = false
    ## start_block is the index of the start of either an unbolded
    ## segment of text or a bolded segment.
    start_block = 0
    for index in range(len(text)):
        ## Handle a starting of bold text
        if text[index] == "<B>":
            if is_bold:
                print("Error: Extra Bold")
            ## print "Not Bold:", text[start_block:index]
```